



A반 4팀

synQ : sync cue

사용자 위치 기반의 중/고음 강조 기능을 갖춘 실시간 동기화 다중 오디오 재생 장치 개발

보고 순서

이 부분에는 해당 타이틀과 관련된 상세 내용을 간략하게 입력해 주세요.

01

개발 **목표**

02

핵심 **기술**

03

세부 개발 내용

04

결과 분석

05

시연 **영상**

06

추후 **계획 및 후기**

개발 목표

“ KEY OBJECTIVE

Wireless Local Network 구축

Raspberry Pi 내장 WiFi를 통하여 보드 간 무선 통신 환경 구축.

Multiple Audio Synchronize

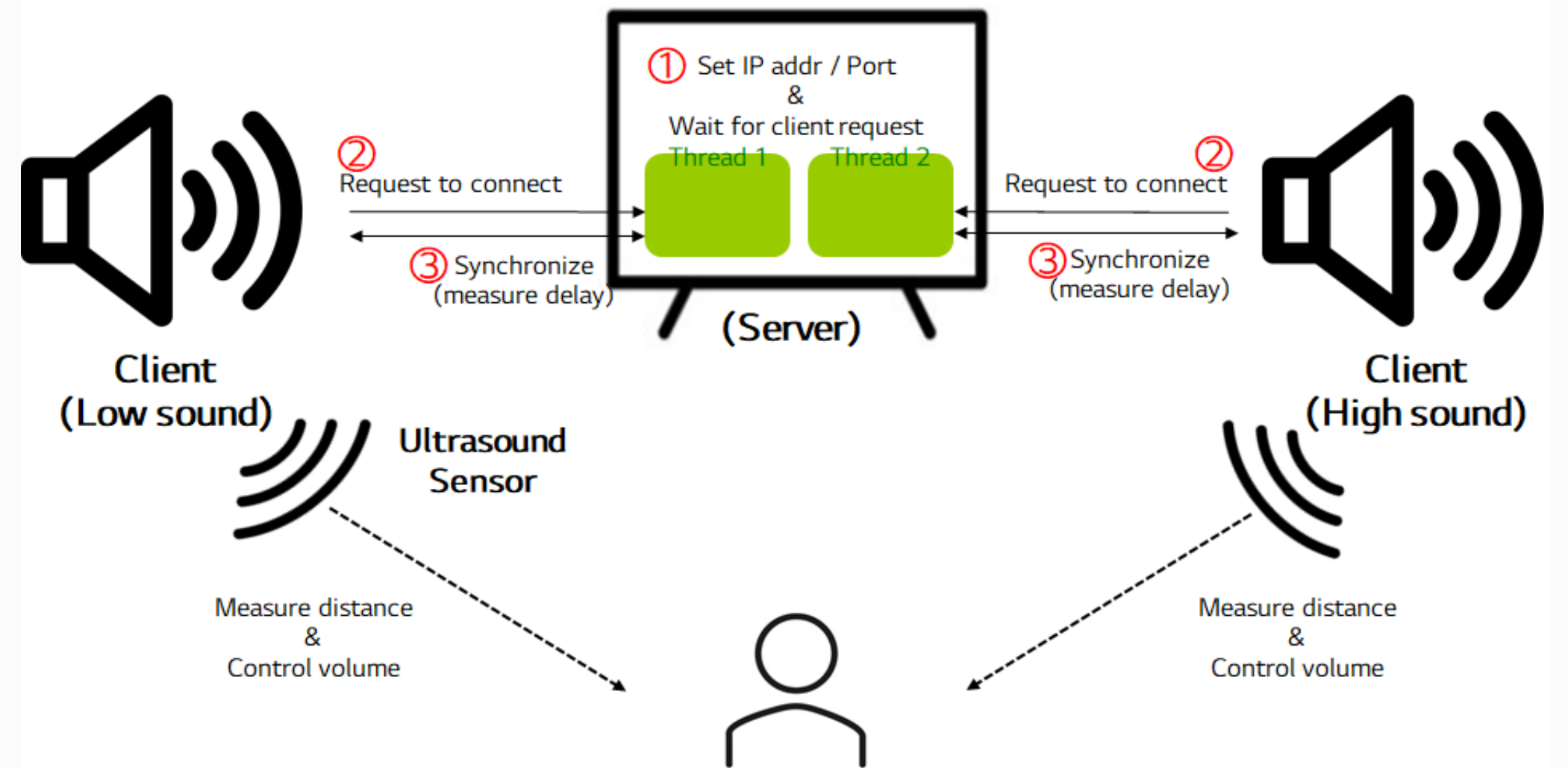
다중 오디오 환경에서 음원이 시차 없이 재생되도록 동기화.

사용자 위치 기반 중/고음 음량 제어

초음파 센서를 이용해 사용자의 거리를 파악하여 중/고음 음량 제어.

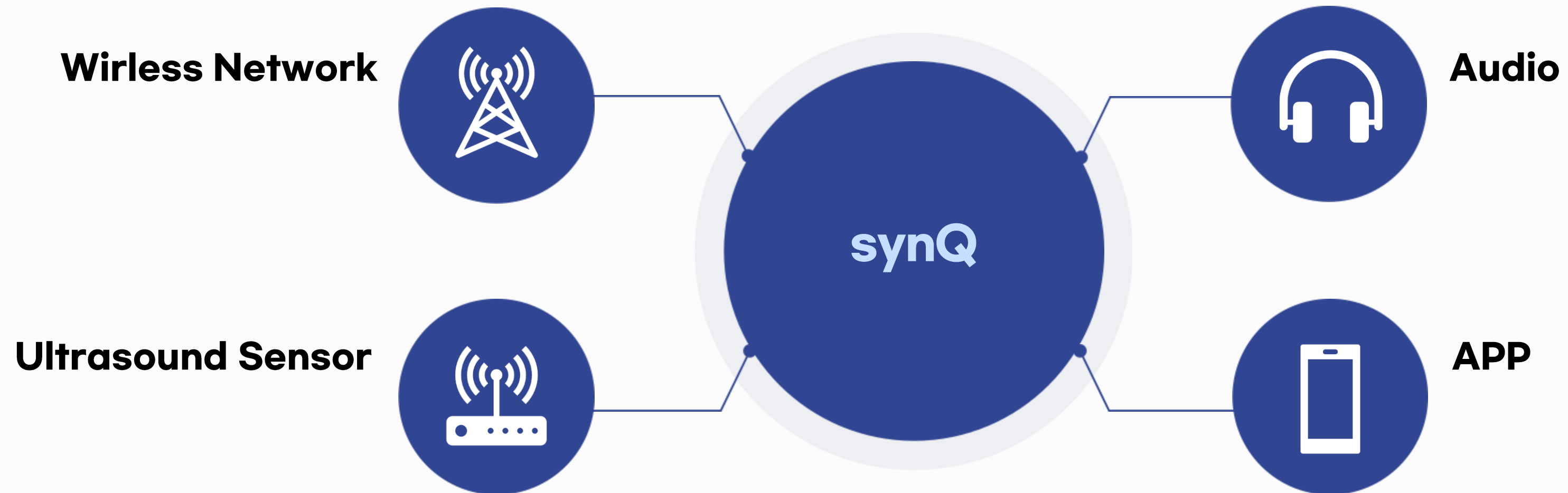
개발 목표

“
KEY
OBJECTIVE



핵심 기술

"TOWARDS A CORE GOAL"



세부 개발 내용

01 Wireless Network

STEP 1: Raspberry Pi 3 내장 Wi-Fi 활성화를 위한 설정

- 기존에는 wlan0 interface가 사용 불가능한 상태

```
# ip link list
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
   link/loopback 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   link/ether b8:27:eb:9d:e5:c8 brd ff:ff:ff:ff:ff:ff
```

As-Is

```
# ip link list
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
   link/loopback 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   link/ether b8:27:eb:9d:e5:c8 brd ff:ff:ff:ff:ff:ff
3: wlan0: <BROADCAST,MULTICAST> mtu 1500
   link/ether b8:27:eb:24:31:a1 brd ff:ff:ff:ff:ff:ff
```

To-Be

세부 개발 내용

01 Wireless Network

STEP 1 : Raspberry Pi 3 내장 Wi-Fi 활성화를 위한 설정

Kernel

- **cfg80211**: IEEE 802.11 표준 무선랜 설정 API 프레임워크, 외부에서 network 제어 가능하게 함
- **MAC80211**: IEEE 802.11 표준의 일부 기능을 driver가 아닌 host CPU 측에서 제어하도록 설정
- **BRCMFMAC**: Broadcom 사의 FullMAC 칩셋을 위한 WiFi 드라이버 사용 설정
- **BRCMFMAC-SDIO**: BRCMFMAC 드라이버가 SDIO 버스 인터페이스를 통해 칩과 통신하도록 설정
- **BRCMSMAC 해제**: Broadcom 사의 드라이버가 SoftMAC으로 작동하지 않도록 함
- **CONFIG_FIRMWARE_LOADER**: FW 로딩 기능 활성화
- **RF switch support**: rfkill과 같은 툴을 통해 무선 장치를 소프트웨어적으로 활성 / 비활성화 할 수 있도록 설정
- **EXTRA_FIRMWARE / EXTRA_FIRMWARE_DIR**: 라즈베리파이 내장 wifi driver에 필요한 펌웨어를 넣어둔 PATH를 설정
- **MMC/SD/SDIO->BCM2835SDHOST** -> WiFi 칩이 연결된 SDIO 버스 자체를 제어하기 위한 설정

DTB

- @wifi 에서 nvram-path / firmware path 지정

세부 개발 내용

01 Wireless Network

STEP 1 : Raspberry Pi 3 내장 Wi-Fi 활성화를 위한 설정

RFS

- **Networking applications**
 - Dnsmasq : DHCP 설정을 위함
 - wpa_supplicant -> wpa_cli / AP mode / WPS / nl80211 support : WiFi 기능을 사용하기 위함
 - wireless-regdb -> 국가별 WiFi 규약 db를 사용하기 위함
- **System tools**
 - Rfkill : software로 무선 기능 power 제어를 위함
- **Firmware**
 - BRCMFMAC-sdio-firmware-rpi
 - BRCMFMAC-sdio-firmware-rpi-wifi

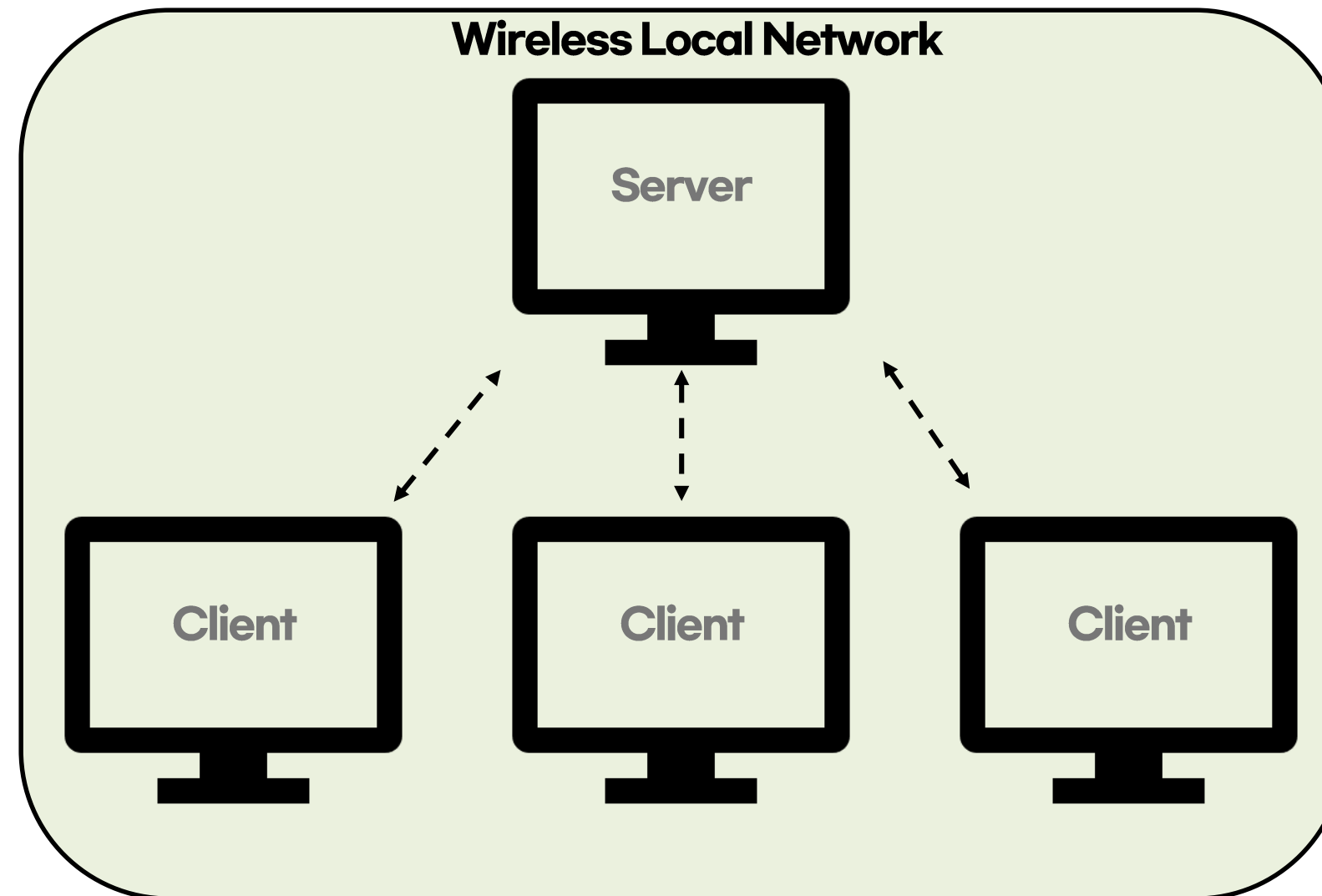
Driver를 User level에서
제어하기 위한 M/W, FW들

세부 개발 내용

01 Wireless Network

STEP 2: Local Network 구성

- Raspberry Pi 보드 간 통신을 위한 **Wireless Local Network** 구성이 필요함
- **Server - Multiple client** 구조이므로 **Server** 보드를 **AP** 모드로 사용하여 NW 구성



세부 개발 내용

01 Wireless Network

STEP 2: Local Network 구성

- AP mode로 NW 구성을 위한 세부 동작

Server

1. wpa_supplicant.conf 수정 (AP mode)
2. IP / netmask 설정
3. wpa_supplicant 구동
4. 음원 송신 program 구동

Client

1. wpa_supplicant.conf 수정 (client)
2. wpa_supplicant 구동
3. DHCP 할당
4. Network scan / connect
5. 음원 수신 program 구동

세부 개발 내용

01 Wireless Network

STEP 2: Local Network 구성

- AP mode로 NW 구성을 위한 세부 설정

Server

- wpa_supplicant.conf

```
ctrl_interface=/var/run/wpa_supplicant
network={
    ssid="myAP"
    mode=2
    key_mgmt=WPA-PSK
    psk="12345678"
    proto=RSN
    frequency=2437
}
```

- cmd

```
ifconfig wlan0 up
ifconfig wlan0 192.168.4.1 netmask 255.255.255.0
wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
```

Client

- wpa_supplicant.conf

```
ctrl_interface=/var/run/wpa_supplicant
update_config=1
network={
    ssid="myAP"
    psk="12345678"
    key_mgmt=WPA-PSK
}
```

- cmd

```
ifconfig wlan0 up
wpa_supplicant -B -i wlan0 -c
wpa_cli -iwlan0 scan
wpa_cli -iwlan0 select_network 0
udhcpc -i wlan0
```

세부 개발 내용

02

Audio

STEP 1 : Multi-Audio Synchronizing

- Open source Snapcast의 Synchronizing
 - 시계 동기화: Time 메시지를 주기적으로 교환하여 모든 클라이언트가 서버와 동일한 '가상 시계'를 갖게 합니다.
 - 타임스탬프 기반 재생: 서버가 보내는 오디오 조각에 재생할 시간을 찍어 보내고 클라이언트들은 동기화된 시계를 기준으로 맞춰 오디오를 재생

01

클라이언트 접속

- 1: 클라이언트가 서버에 연결을 요청
- 2: 클라이언트는 Hello 메시지를 전송
- 3: 서버는 Server Settings 메시지와 Codec Header 메시지를 전송

02

Time Synchronization

- 1: 클라이언트는 주기적으로 서버에게 Time 메시지를 전송
- 2: 서버는 메시지를 받아 다시 클라이언트에게 응답하고 걸린 시간을 계산
- 3: 이 시간 차이를 바탕으로 클라이언트는 자신의 내부 시계를 서버의 시계에 맞춤

03

오디오 데이터 전송

- 1: 서버는 오디오 스트림을 아주 작은 조각으로 잘라서 Wire Chunk라는 메시지 전송
- 2: 각 오디오 조각(Wire Chunk)에는 언제 재생되어야 하는지(timestamp)가 포함
- 3: 클라이언트는 앞서 동기화된 자신의 시계를 기준으로, 이 timestamp에 맞춰 재생합니다.

세부 개발 내용

02

Audio

STEP 1 : Multi-Audio Synchronizing

- Open source Snapcast 각 메시지 타입 정리

Hello

Field	Type	Description
size	uint32	Size of the following JSON string
payload	char[]	JSON string containing the message (not null terminated)

Server Settings

Field	Type	Description
size	uint32	Size of the following JSON string
payload	char[]	JSON string containing the message (not null terminated)

Time

Field	Type	Description
latency.sec	int32	The second value of the latency between the server and the client
latency.usec	int32	The microsecond value of the latency between the server and the client

Wire Chunk

Field	Type	Description
timestamp.sec	int32	The second value of the timestamp when this part of the stream was recorded
timestamp.usec	int32	The microsecond value of the timestamp when this part of the stream was recorded
size	uint32	Size of the following payload
payload	char[]	Buffer of data containing the encoded PCM data (a decodable chunk per message)

Codec Header

Field	Type	Description
codec_size	uint32	Length of the codec string (not including a null character)
codec	char[]	String describing the codec (not null terminated)
size	uint32	Size of the following payload
payload	char[]	Buffer of data containing the codec header

세부 개발 내용

02

Audio

STEP 2: Cross-Compiling Snapcast for Raspberry Pi (arm64)

- 빌드 환경과 타겟 환경의 차이 -> Architecture mismatch, glibc version incompatibility
 - Linux Ubuntu 20.04 (x86_64, glibc 2.31)
 - Raspberry Pi (arm64, glibc 2.25)



도커 환경

- 1: Raspberry Pi 의 glibc 2.25 환경에서 호환되는 빌드 환경 필요
- 2: arm64 아키텍처의 Debian 9 (Stretch) 도커 이미지 사용
- 3: -v \$(pwd):/workspace: 현재 디렉토리를 컨테이너 내부 폴더로 mount.

```
root@2d55e16e8473:~# cd /src/snapcast/
root@2d55e16e8473:/src/snapcast# ls
CMakeLists.txt  Doxyfile  README.md  _CMakePresets.json  alsa  build  client  common
doc  libs  test  tools
CONTRIBUTING.md  LICENSE  TODO.md  _config.yml  bin  changelog.md  cmake  control  extras
server  test-music.mp3
root@2d55e16e8473:/src/snapcast#
```

세부 개발 내용

02

Audio

STEP 3: Installing dependency libraries within a Docker container

- 패키지 매니저를 통한 기본 의존성 설치
 - Apt 패키지 목록을 업데이트 후 필요 라이브러리 설치
- 수동 설치가 필요한 라이브러리 (Cmake, Boost 등)
 - Debian 9의 apt로 제공 되는 버전은 Snapcast 요구사항 보다 낮음 -> 소스 받은 후 컴파일

```
# 컨테이너 내부에서 실행
apt-get install -y wget
wget https://github.com/Kitware/CMake/releases/download/v3.25.0/cmake-3.25.0.tar.gz
tar -zxvf cmake-3.25.0.tar.gz
cd cmake-3.25.0
./bootstrap
make
make install
cmake --version # 설치된 버전 확인
cd ..
```

```
# 컨테이너 내부에서 실행
wget https://boostorg.jfrog.io/artifactory/main/release/1.81.0/source/boost_1_81_0.tar.gz tar -
zxf boost_1_81_0.tar.gz
```

세부 개발 내용

02

Audio

STEP 4 : Snapserver, Snapclient dependency libraries

- Ldd 를 통한 Snapserver Snapclient 필요 라이브러리 확인
 - 오디오 코덱(Opus, Vorbis, FLAC), 네트워크 서비스 탐색(Avahi), 그리고 C/C++ 표준 라이브러리 등 대부분의 핵심 라이브러리를 공유
 - snapserver는 오디오를 압축하기 위한 libvorbisenc 같은 인코딩 라이브러리
 - snapclient는 ALSA(libasound) 라이브러리를 통해 실제 오디오 출력에 더 중점
- .so 파일 RFS user/lib 추가를 위한 폴더 생성

```
root@2d55e16e8473:/src/snapcast/bin# ldd snapserver
libpthread.so.0 => /lib/aarch64-linux-gnu/libpthread.so.0
libavahi-common.so.3 => /usr/lib/aarch64-linux-gnu/libavahi-common.so.3
libavahi-client.so.3 => /usr/lib/aarch64-linux-gnu/libavahi-client.so.3
libvorbis.so.0 => /usr/lib/aarch64-linux-gnu/libvorbis.so.0
liblz4.so.1 => /usr/lib/aarch64-linux-gnu/liblz4.so.1 (0x00000000)
....
libgcrypt.so.20 => /lib/aarch64-linux-gnu/libgcrypt.so.20
libpcrc.so.3 => /lib/aarch64-linux-gnu/libpcrc.so.3 (0x00000000)
libgpg-error.so.0 => /lib/aarch64-linux-gnu/libgpg-error.so.0
root@2d55e16e8473:/src/snapcast/bin#
```

```
root@2d55e16e8473:/src/snapcast/bin# ldd snapclient
libpthread.so.0 => /lib/aarch64-linux-gnu/libpthread.so.0
libavahi-common.so.3 => /usr/lib/aarch64-linux-gnu/libavahi-common.so.3
libavahi-client.so.3 => /usr/lib/aarch64-linux-gnu/libavahi-client.so.3
libasound.so.2 => /usr/lib/aarch64-linux-gnu/libasound.so.2
...
libssl.so.1.1 => /usr/lib/aarch64-linux-gnu/libssl.so.1.1
libpcrc.so.3 => /lib/aarch64-linux-gnu/libpcrc.so.3 (0x00000000)
libgpg-error.so.0 => /lib/aarch64-linux-gnu/libgpg-error.so.0
root@2d55e16e8473:/src/snapcast/bin#
```


세부 개발 내용

02

Audio

STEP 4 : Snapserver, Snapclient dependency libraries

Common

- **libc.so.6**: C 표준 라이브러리 (Glibc): 리눅스 시스템의 기본적인 라이브러리. 파일 입출력, 메모리 관리 등 모든 프로그램의 근간이 되는 기능을 제공
- **libstdc++.so.6**: C++로 작성된 프로그램에 필수적인 라이브러리. 클래스, 객체, 템플릿 등 C++ 핵심 기능을 지원
- **libpthread.so.0**: POSIX 스레드 라이브러리: 멀티스레딩을 지원
- **libopus.so.0**: Opus 코덱 라이브러리: 고품질, 저지연 오디오 압축 코덱인 Opus를 디코딩/인코딩하는 데 사용
- **libvorbis.so.0**: Vorbis 코덱 라이브러리: Ogg Vorbis 오디오 형식을 처리하는 라이브러리
- **libFLAC.so.8**: FLAC 코덱 라이브러리: 무손실 오디오 압축 코덱인 FLAC을 처리하기 위한 라이브러리
- **libssl.so.1.1**: OpenSSL (SSL/TLS) 라이브러리: (선택적) Snapcast 스트림을 암호화하여 보안 통신을 제공하는 데 사용

Snapserver

- **libvorbisenc.so.2**: Vorbis 인코더 라이브러리, Ogg Vorbis 형식으로 **압축(인코딩)**하여 클라이언트로 스트리밍할 때 사용
- **libexpat.so.1**: Expat XML 파서 라이브러리

Snapclient

- **libasound.so.2**: 리눅스의 고급 사운드 아키텍처(ALSA)를 통해 실제 사운드 카드로 오디오 데이터를 보내 재생하는 역할

세부 개발 내용

02

Audio

STEP 5 : SnapCast Customizing

- 기존 Open source Snapcast에 존재하지 않는 음역대 분리 및 다중 포트 파이프 구현
- ALSA 라이브러리 및 FFmpeg 유틸리티를 커널 및 RFS 적용
- FFmpeg 유틸리티를 통해 250Hz 기준 저음역대 및 고음역대 분리
- 저음역대, 고음역대, 원본 세 파이프라인을 각각 다른 포트로 구동 (1704, 1705, 1706)
- TCP 통신으로 클라이언트에서 오디오 청크를 가져가는 방식으로 구동

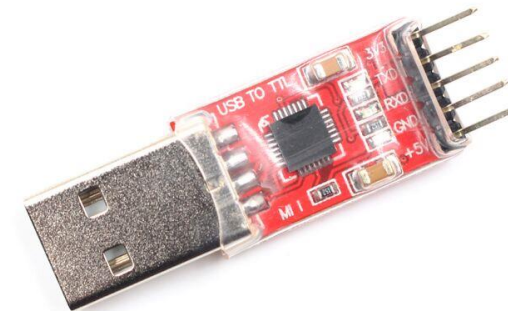
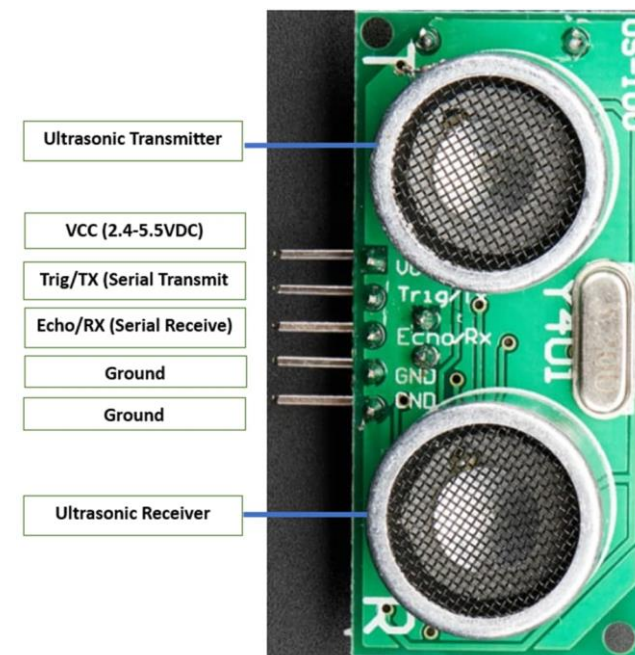
```
startProc(procSnapLow, "/tmp/snap_low", 1780, 1707, 1704,  
          &MainWindow::onSnapOutputLow, lblClient1);  
startProc(procSnapHigh, "/tmp/snap_high", 1781, 1708, 1705,  
          &MainWindow::onSnapOutputHigh, lblClient2);  
startProc(procSnapOrigin, "/tmp/snap_origin", 1782, 1709, 1706,  
          &MainWindow::onSnapOutputOrigin, lblClient3);
```

세부 개발 내용

03 Ultrasound Sensor

STEP1: 초음파 센서를 통한 거리 감지 시스템

- US-100 Ultrasound Sensor
- 개발 편의 및 확장을 위해 GPIO 핀 제어가 아닌 UART 제어 도입
- CP2102 UART-USB Gender 이용하여 데이터 수신
- 초음파 센서를 통해 측정된 거리에 따른 데시벨 제어 시스템

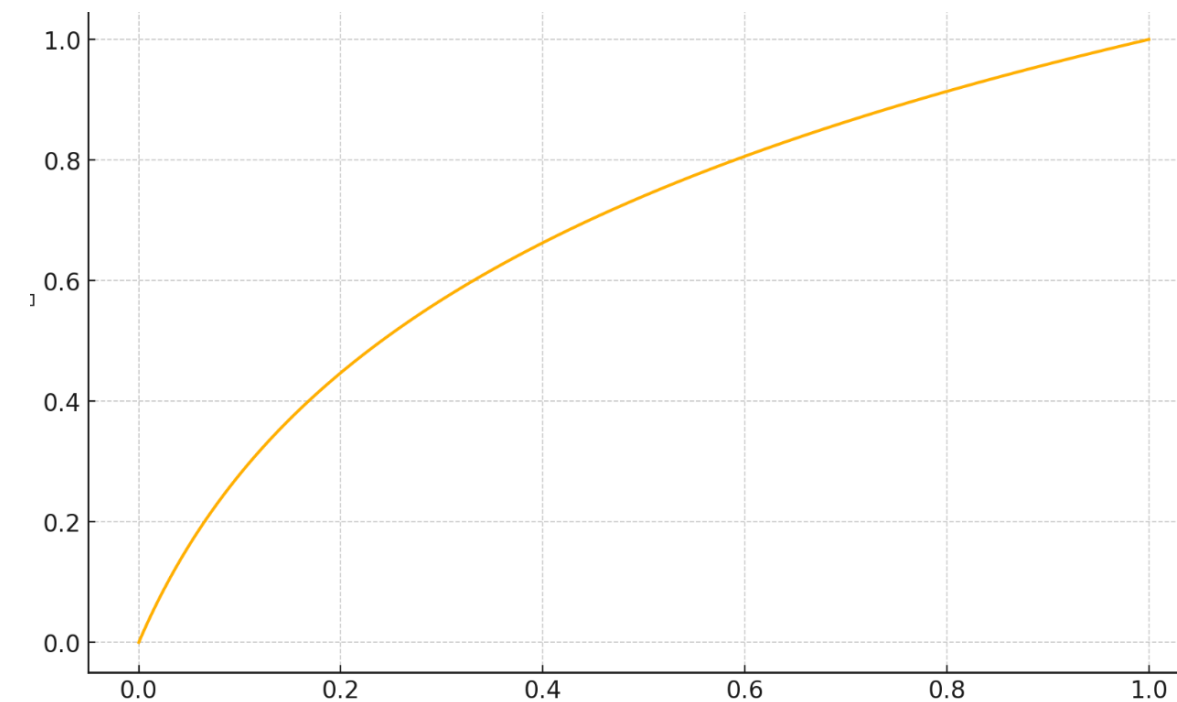
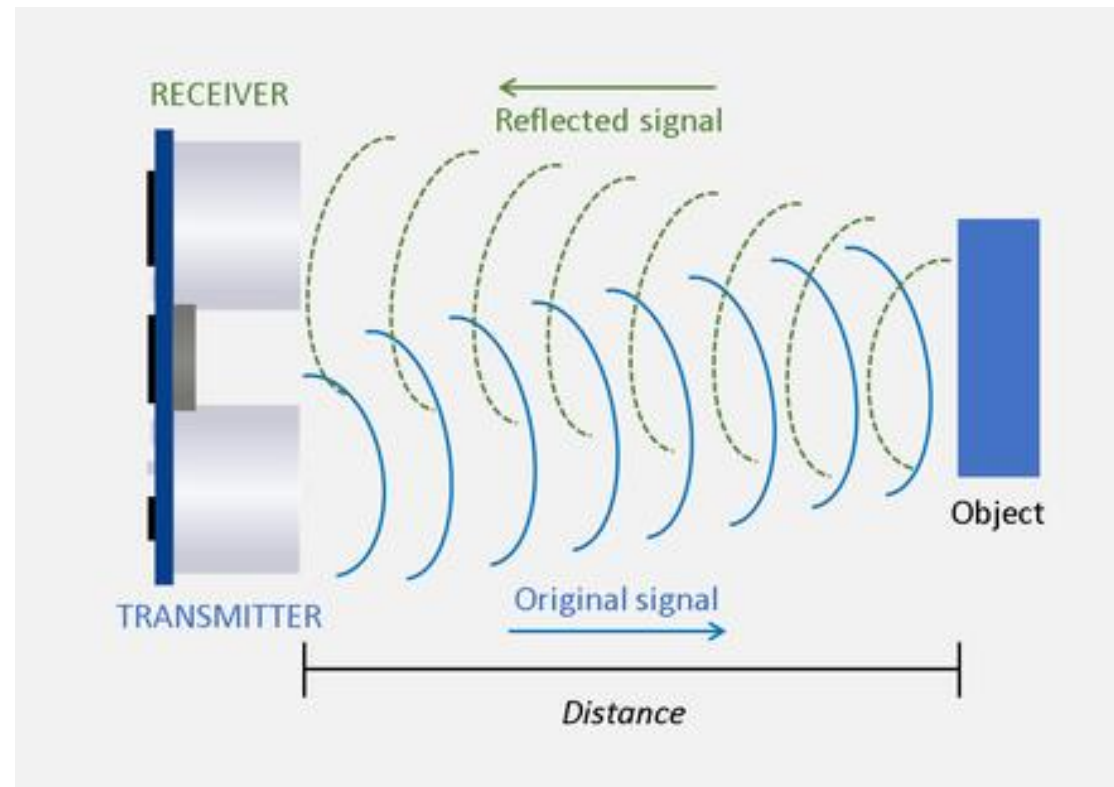


세부 개발 내용

03 Ultrasound Sensor

STEP 2: 로그 커브 기반 제어

- 입력된 Distance를 클램프 후 0~1 사이 x로 정규화
- 정규화된 x에 대해 $\ln(1+kx) / \ln(1+k)$ 로 Scaling 보정
- 초반 거리 증가에 대한 가중치 부여



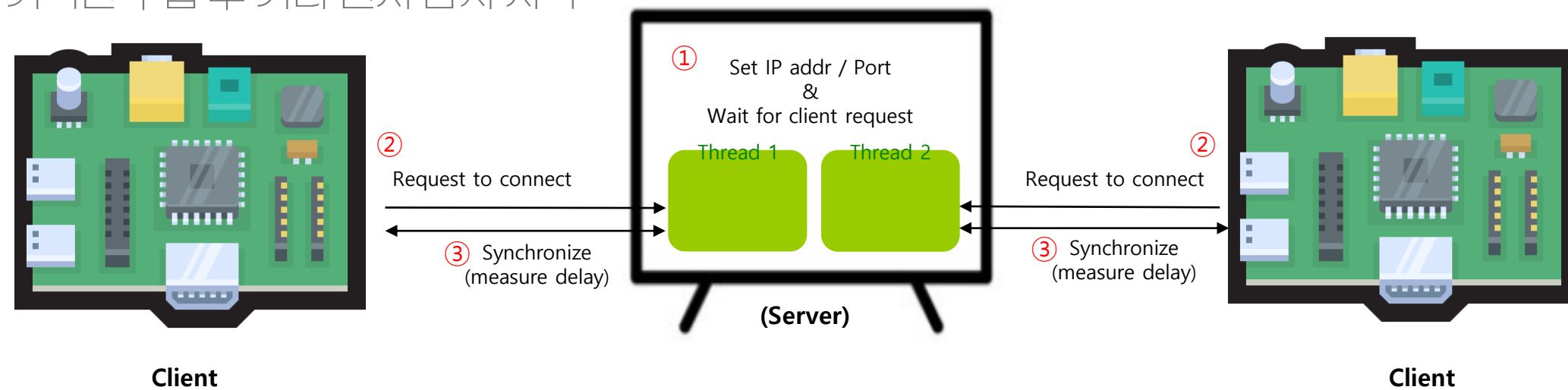
세부 개발 내용

04

APP

STEP1: APP 자동 실행 및 컨넥션 수립

- Client와 Server 개별 RFS 구축
- APP 실행 및 snapcast connect 수립 자동화
- 하드웨어 MAC 주소를 기반으로 연결 수립 확인
- 커넥션 수립 후 거리 센서 감지 시작



결과 분석

1

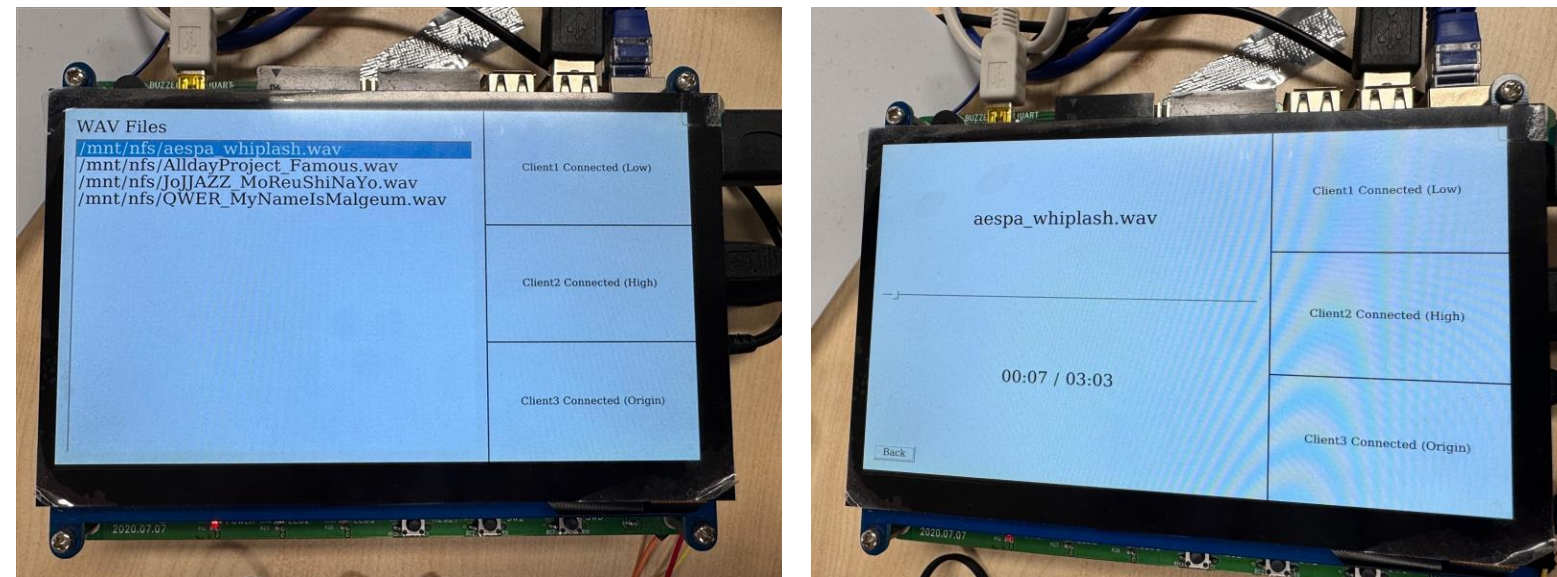


성과 달성률

1. 무선 Local NW 구성 완료
2. Audio 동기화 완료
3. 음원 채널 분리 및 음량제어 완료
4. 초음파 센서 제어 완료
5. APP 시각화 미흡

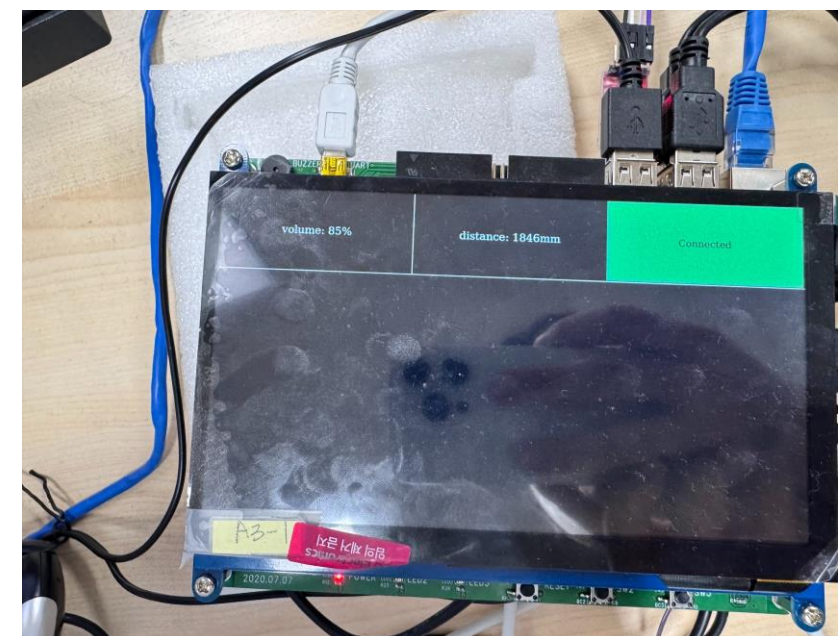
2

APP 화면 (Server)



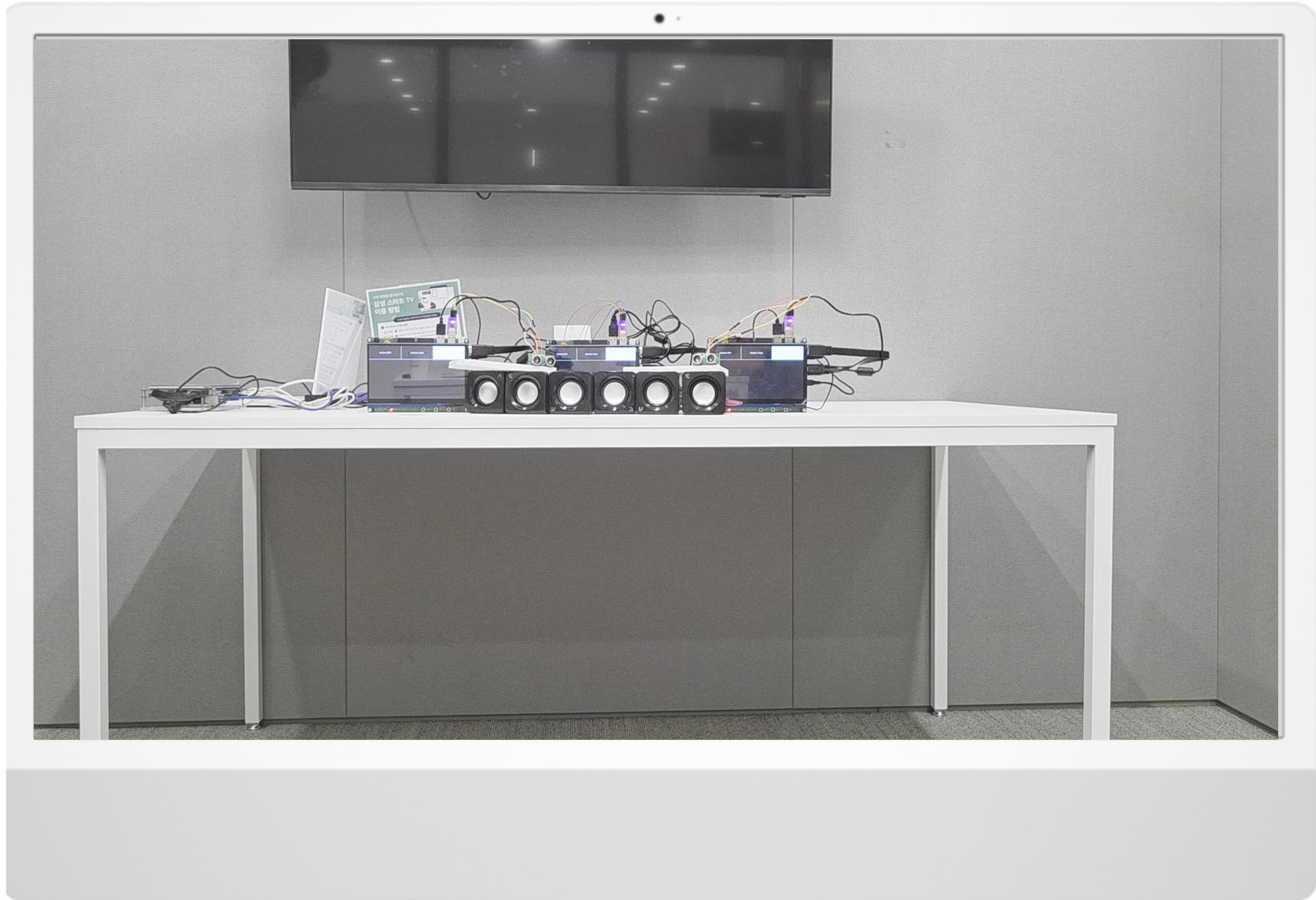
3

APP 화면 (Client)



이 부분에는 해당 타이틀과 관련된 상세 내용을 간략하게 입력해 주세요.

시연영상



추후 계획 및 후기

추후 개선점

1. 5GHz 지원 WiFi 모듈 사용
2. 외장 WiFi 모듈 사용
3. Connection 복구 로직 개선
4. UI 고도화

후속 개발 주제

1. 사용 환경에 따른 최적화 마법사 지원
2. 다양한 스피커 드라이버 지원
3. 음역대 분리 세밀화

요약

장비 및 환경 제약으로 구현하지 못한 것들에 대한 추후 도전 가능

추후 계획 및 후기

팀장

김주현 연구원

프로그램을 만들어서 배포하는 과정이 생각보다 일이 많이 터진다는 것을 깨달았습니다. 다음부터는 이슈 보는 기간도 추가하여 계획을 짜겠습니다.

정지우 연구원

APP구현이 생각보다 복잡해서 더 배워야 할 것 같습니다.

정현오 연구원

당연히 된다고 생각했던 주변 기기들이 사실은 다루기 정말 어렵다는 것을 깨달았습니다. 많은 것을 배워서 행복합니다. 엘지전자 파이팅!

홍민기 연구원

오픈 소스 크로스 컴파일을 프로젝트에서 활용해보며 저희 webOS에서 AI 관련 오픈 소스를 포팅하여 적극적으로 활용할 수 있는 발판을 만들 수 있어 좋은 경험이었습니다.

THANK YOU

감사합니다

프로젝트 기간 : 2025년 7월 1일 ~ 7월 9일



김주현 연구원 / MS

- 초음파 센서
- 음원 채널 분리
- APP / RFS



정현오 연구원 / MS

- Wi-Fi
- Local NW 제어



정지우 연구원 / VS

- App



홍민기 연구원 / MS

- Audio Synchronize
- Open source porting

프로젝트 멤버

김주현 연구원 (MS) / 정지우 연구원 (VS) / 정현오 연구원 (MS) / 홍민기 연구원 (MS)